

BACHELOR OF SCIENCE IN CYBER SECURITY

CYS 121 – Information Systems Security

Laboratory Manual – Unit 1: User Authentication

Student Name	Incognito
Student Number	2222222222
Course Code	CYS 121
Unit Title	Unit 1: User Authentication
Lecturer	Dr. Lungu Nelson
Submission Date	07 April 2026
Platform	Ubuntu 24.04 LTS (WSL2 / VirtualBox)

Fully Self-Guided Practical Laboratory Exercise

1. Introduction

This laboratory report documents the practical work completed for CYS 121 Information Systems Security, Unit 1: User Authentication. The exercise was performed on an Ubuntu Linux environment (Windows Subsystem for Linux 2 / VirtualBox) and covered four core areas: creating and managing local user accounts, enforcing a stronger password policy, configuring Secure Shell (SSH) key-based authentication, and analysing authentication factors in the context of real-world information systems.

User authentication is the fundamental mechanism by which an information system verifies that a person attempting to gain access is who they claim to be. Weak or misconfigured authentication is consistently cited as one of the top causes of data breaches. This lab provides hands-on experience configuring authentication controls directly on a Linux system, building practical skills directly relevant to a career in cyber security.

2. Learning Outcomes Addressed

- Create and manage local Linux user accounts in a controlled training environment.
- Apply a stronger password policy and test the difference between weak and strong passwords.
- Configure Secure Shell key-based login for localhost as a safe simulation of remote authentication.
- Differentiate knowledge, possession, inherence, and multi-factor authentication using practical examples.

3. Environment Preparation

Before beginning the main practical tasks, the following preparation steps were completed to ensure a stable and reproducible working environment.

3.1 System Details

Parameter	Value
Operating System	Ubuntu 24.04 LTS (WSL2 / VirtualBox)
Terminal	Bash shell – opened with Ctrl+Alt+T
Date of Lab	07 April 2026
Hostname	[output of: hostname]
IP Address	[output of: hostname -I]
Working Directory	~/CYS121_Labs/unit1_authentication

3.2 Preparation Commands Executed

The following commands were executed in sequence to prepare the environment:

```
ev4ns@DESKTOP-K7CKH77:~$ mkdir -p ~/CYS121_Labs/unit1_authentication
ev4ns@DESKTOP-K7CKH77:~$ cd ~/CYS121_Labs/unit1_authentication
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ pwd
/home/ev4ns/CYS121_Labs/unit1_authentication
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ apt install -y openssh-server libpam-pwquality
E: Could not open lock file /var/lib/dpkg/lock-frontent - open (13: Permission denied)
E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), are you root?
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ sudo apt install -y openssh-server libpam-pwquality
[sudo] password for ev4ns:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssh-server is already the newest version (1:9.6p1-3ubuntu13.15).
The following additional packages will be installed:
  cracklib-runtime libcrack2 libpwquality-common libpwquality1 wamerican
The following NEW packages will be installed:
  cracklib-runtime libcrack2 libpam-pwquality libpwquality-common libpwquality1 wamerican
0 upgraded, 6 newly installed, 0 to remove and 57 not upgraded.
Need to get 446 kB of archives.
```

```
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-04-07 21:28:55 CAT; 2h 27min ago
 TriggeredBy: ● ssh.socket
    Docs: man:sshd(8)
          man:sshd_config(5)
   Process: 262 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 283 (sshd)
    Tasks: 1 (limit: 4615)
   Memory: 2.0M (peak: 2.5M)
      CPU: 88ms
   CGroup: /system.slice/ssh.service
           └─283 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Apr 07 21:28:54 DESKTOP-K7CKH77 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Apr 07 21:28:55 DESKTOP-K7CKH77 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Apr 07 21:28:55 DESKTOP-K7CKH77 sshd[283]: Server listening on 0.0.0.0 port 22.
Apr 07 21:28:55 DESKTOP-K7CKH77 sshd[283]: Server listening on :: port 22.
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ date && hostname && hostname -I
Tue Apr  7 23:57:57 CAT 2026
DESKTOP-K7CKH77
172.25.99.242 172.17.0.1
```

The SSH service was confirmed as active (running), meaning the OpenSSH server was available for use in Part C. The package manager successfully installed both openssh-server and libpam-pwquality without errors.

4. Part A – Creating User Accounts and Inspecting Authentication Files

4.1 Procedure

Two test user accounts were created using the adduser command. This command automatically creates a home directory, copies skeleton files, and prompts for a password and optional profile information. The profile prompts were skipped by pressing Enter as they are not relevant to this exercise.

Commands executed:

```
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ sudo adduser teller1
info: Adding user `teller1' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `teller1' (1007) ...
info: Adding new user `teller1' (1007) with group `teller1 (1007)' ...
info: Creating home directory `/home/teller1' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
Retype new password:
passwd: password updated successfully
Changing the user information for teller1
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
info: Adding new user `teller1' to supplemental / extra groups `users' ...
info: Adding user `teller1' to group `users' ...

ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ id teller1
uid=1007(teller1) gid=1007(teller1) groups=1007(teller1),100(users)
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ id auditor1
uid=1008(auditor1) gid=1008(auditor1) groups=1008(auditor1),100(users)
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ ls -l /etc/passwd /etc/shadow
-rw-r--r-- 1 root root  2074 Apr  8 00:04 /etc/passwd
-rw-r----- 1 root shadow 1453 Apr  8 00:04 /etc/shadow
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ grep 'teller1|auditor1' /etc/passwd
teller1:x:1007:1007:,,,:/home/teller1:/bin/bash
auditor1:x:1008:1008:,,,:/home/auditor1:/bin/bash
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ grep -E 'teller1|auditor1' /etc/passwd
teller1:x:1007:1007:,,,:/home/teller1:/bin/bash
auditor1:x:1008:1008:,,,:/home/auditor1:/bin/bash
```

4.2 Results and Observations

The id command confirmed that other accounts were created successfully. The output for each user included a unique User ID (UID), primary Group ID (GID), and group membership information.

4.3 Interpretation of /etc/passwd and /etc/shadow

The /etc/passwd file is world-readable (permissions -rw-r--r--) and stores basic account information. Each line contains seven colon-delimited fields:

- Username (e.g., teller1)
- Password placeholder – the x indicates the actual hash is stored in /etc/shadow
- User ID (UID) – a unique numeric identifier (1001)
- Group ID (GID) – the primary group
- GECOS field – optional descriptive information (name, phone, etc.)
- Home directory (e.g., /home/teller1)
- Login shell (e.g., /bin/bash)

The /etc/shadow file has much tighter permissions (-rw-r-----), readable only by root and the shadow group. It stores the salted and hashed password for each account, along with password ageing information such as last-change date, minimum and maximum password age, and account expiry. Separating the hash into shadow prevents unprivileged users from obtaining hashes for offline cracking attempts.

5. Part B – Applying and Testing a Stronger Password Policy

5.1 Procedure

Before modifying system configuration, a backup of the original password quality configuration file was created. This is good practice: if a mistake is introduced during editing, the original can be restored quickly.

```
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ cp /etc/security/pwquality.conf ~/CYS121_Labs/unit1_authentication/pwquality.conf.
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ sudo nano /etc/security/pwquality.conf
```

The following four settings were appended to the bottom of the file:

Setting	Value	What It Enforces
minlen	10	Password must be at least 10 characters long
minclass	3	Must contain at least 3 of: uppercase, lowercase, digits, symbols
maxrepeat	3	No character may appear more than 3 times in a row
retry	3	User gets 3 attempts before the passwd command exits

```
# Skip testing the password quality for users that are not present in the
# /etc/passwd file.
# Enabled if the option is present.
# local_users_only

minlen = 10
minclass = 3
maxrepeat = 3
retry = 3
```

After saving and exiting the editor (Ctrl+O, Enter, Ctrl+X), the settings were verified with:

```
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ tail -n 8 /etc/security/pwquality.conf
# /etc/passwd file.
# Enabled if the option is present.
# local_users_only

minlen = 10
minclass = 3
maxrepeat = 3
retry = 3
```

Password testing was then performed from inside each test account:

- Start by switching account

```
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ su - teller1
Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Apr  8 00:25:26 CAT 2026

System load:  0.16                       Processes:    52
Usage of /:   1.6% of 1006.85GB          Users logged in:  1
Memory usage: 11%                       IPv4 address for eth0: 172.25.99.242
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/teller1/.hushlogin file.
teller1@DESKTOP-K7CKH77:~$ passwd

teller1@DESKTOP-K7CKH77:~$ passwd
Changing password for teller1.
Current password:
New password:
BAD PASSWORD: The password contains less than 3 character classes
New password:
```

5.2 Results – Password Policy Testing

When a weak password such as school123 was entered, the PAM (Pluggable Authentication Modules) subsystem invoked pam_pwquality and reported an error similar to:

```
BAD PASSWORD: The password fails the dictionary check - it is
based on a dictionary word
BAD PASSWORD: The password is too simple
```

A strong password containing all required character classes (e.g., Secure@Lab2024!) was accepted without warnings. The policy enforcement confirmed that pam_pwquality was correctly integrated into the PAM stack on the Ubuntu system.

5.3 Security Analysis – Why Stronger Policies Matter

Password policies directly reduce the probability that an account can be compromised through common attack methods:

- Dictionary attacks: Requiring mixed character classes means simple words and common substitutions (p@ssw0rd) are rejected, forcing attackers to work through a vastly larger search space.
- Brute-force attacks: A minimum length of 10 characters increases the keyspace exponentially compared to a 6- or 8-character policy.
- Credential stuffing and reuse: maxrepeat prevents trivially constructed passwords (e.g., aaa12345) that appear frequently in leaked credential databases.
- Guessing: Combining length and complexity requirements means even a targeted attacker who knows personal information about the account holder cannot reliably guess the password within a reasonable number of attempts.

From a compliance perspective, policies aligned with NIST SP 800-63B, ISO/IEC 27001, and PCI DSS require enforced complexity or length minimums. The configuration performed in this part demonstrates how a Linux administrator implements those controls at the system level.

6. Part C – Configuring SSH Key-Based Authentication

6.1 What is SSH Key-Based Authentication?

Secure Shell (SSH) is a cryptographic network protocol for operating network services securely. Instead of a password, key-based authentication uses an asymmetric key pair:

- Private key – kept secret on the client machine. It is never transmitted over the network.
- Public key – copied to the server's authorized_keys file. Anyone may hold a copy without risk.

When a client connects, the server issues a mathematical challenge that only the holder of the matching private key can answer correctly. This eliminates the risk of password interception, brute-force, and credential stuffing attacks against the SSH service.

6.2 Procedure and Commands

Configure SSH Key-Based Authentication – we start by generating SSH key pair

Step 1 – Generate an Ed25519 key pair (modern, fast, and highly secure):


```

ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ ssh-keygen -t ed25519 -C "cys121-lab"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ev4ns/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ev4ns/.ssh/id_ed25519
Your public key has been saved in /home/ev4ns/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:G+rQX5VQfZMgs+RkIoVt7LB60Dm3zd8vb5wOfjHTDTw cys121-lab
The key's randomart image is:
+--[ED25519 256]--+
|
|   . = o * . o . . |
|   o . + * . + . o . |
|   . * . o . . . |
|   . = o . . E |
|   o S + o . oo |
|   o o + + . +o |
|   . + . . . . . = |
|   o . . . . o +o |
|   . . . . o = + |
+-----[SHA256]-----+

```

When prompted for a save location, the default (~/.ssh/id_ed25519) was accepted. A passphrase was optionally entered to protect the private key at rest.

Step 2 – Copy the public key to the teller1 account on the local machine:

```

ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ ssh-copy-id teller1@localhost
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/ev4ns/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed — if you are prompted now it is to install the new keys
teller1@localhost's password:
Permission denied, please try again.
teller1@localhost's password:
Permission denied, please try again.
teller1@localhost's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'teller1@localhost'"
and check to make sure that only the key(s) you wanted were added.

```

This command authenticated with the teller1 password and appended the public key to /home/teller1/.ssh/authorized_keys with correct permissions.

Step 3 – Test the key-based login:

```
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ ssh teller1@localhost
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Apr  8 00:47:47 CAT 2026

System load:  0.0               Processes:            52
Usage of /:   1.6% of 1006.85GB Users logged in:     1
Memory usage: 12%              IPv4 address for eth0: 172.25.99.242
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge
teller1@DESKTOP-K7CKH77:~$ whoami
teller1
teller1@DESKTOP-K7CKH77:~$ hostname
DESKTOP-K7CKH77
teller1@DESKTOP-K7CKH77:~$ exit
logout
Connection to localhost closed.
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ |
```

Step 4 – Examine the SSH server configuration to note the PasswordAuthentication setting:

```
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ grep -i "PasswordAuthentication" /etc/ssh/sshd_config
#PasswordAuthentication yes
# PasswordAuthentication. Depending on your PAM configuration,
# PAM authentication, then enable this but set PasswordAuthentication
ev4ns@DESKTOP-K7CKH77:~/CYS121_Labs/unit1_authentication$ sudo nano /etc/ssh/sshd_config
[sudo] password for ev4ns:
```

6.3 Results

The ssh teller1@localhost command connected without prompting for the teller1 account password, confirming that key-based authentication was working correctly. The whoami command returned teller1 and hostname returned the correct machine name, proving the session was established as the expected user on the expected system.

Test	Result
ssh-keygen execution	Ed25519 key pair generated; public key saved to ~/.ssh/id_ed25519.pub
ssh-copy-id teller1@localhost	Key appended to /home/teller1/.ssh/authorized_keys successfully

Test	Result
ssh teller1@localhost (key auth)	Connected without password prompt – key authentication successful
whoami inside session	Returned: teller1
hostname inside session	Returned: correct machine hostname
PasswordAuthentication in sshd_config	Set to 'yes' by default; noted but not changed (lab requirement)

6.4 Why Key-Based Login is Stronger than Password-Only Login

Key-based SSH authentication is considered superior to password-based authentication for several important reasons:

- Immunity to brute-force: Ed25519 private keys are 256-bit randomly generated values. There is no dictionary or brute-force attack that can guess a private key within any practical timeframe.
- No credential transmission: The private key never leaves the client machine. A network eavesdropper who captures the entire SSH handshake gains no usable secret.
- Phishing resistance: Passwords can be stolen by fraudulent login pages. A key pair cannot be phished because the private key is never typed by the user.
- Centralised revocation: Removing a public key from authorized_keys immediately revokes access, whereas password resets depend on the user co-operating or the administrator forcing a change.
- Optional passphrase adds a second factor: Protecting the private key file with a passphrase means an attacker who steals the key file still cannot use it without the passphrase – effectively creating a two-factor mechanism (possession + knowledge).

7. Part D – Interpreting Authentication Factors

7.1 Authentication Factor Comparison Table

Factor Type	Definition	Real-World Example	Key Limitation
Knowledge Factor (Something you know)	A secret that only the legitimate user is supposed to know.	Password, PIN, security question answer, passphrase.	Can be guessed, phished, stolen via keylogger, or leaked in a data breach. Users often choose weak or reused secrets.
Possession Factor (Something you have)	A physical or digital object that the user must physically hold or control.	Smart card, hardware security token (YubiKey), one-time password (OTP)	Can be lost, stolen, cloned (card skimming), or left behind. Requires the

Factor Type	Definition	Real-World Example	Key Limitation
		authenticator app, bank card.	user to always carry the object.
Inherence Factor (Something you are)	A biological or behavioural characteristic unique to the individual.	Fingerprint scan, facial recognition, iris scan, voice recognition, gait analysis.	Cannot be changed if compromised. Sensor accuracy depends on conditions (lighting, injury, dirt). Risk of false rejection or false acceptance. Privacy concerns around biometric data storage.
Multi-Factor Authentication (MFA)	Requiring two or more factors from different categories before granting access.	ATM: bank card (possession) + PIN (knowledge). VPN: password (knowledge) + OTP on phone (possession). Laptop: fingerprint (inherence) + password (knowledge).	Inconvenient for users if poorly implemented. SMS-based second factors are vulnerable to SIM-swapping attacks. Recovery options can create backdoors if not secured.

7.2 Biometric Authentication – Benefit and Limitation

Biometric authentication offers a significant usability advantage: the user does not need to remember anything or carry a physical object. This reduces help-desk overhead for forgotten passwords and eliminates the risk of password reuse across multiple systems. In a high-throughput environment such as a hospital or airport, fingerprint or facial recognition authentication also speeds up the login process considerably.

However, biometric systems are not infallible. Physical factors affecting a sensor can cause false rejections: a dirty or wet fingerprint sensor may fail to read a legitimate user's print, a facial recognition camera may misidentify a user in poor lighting or if the user is wearing a mask. An injury such as a cut on the fingertip permanently changes the biometric template until healed. Unlike a password or token, a compromised biometric cannot simply be reset – once a fingerprint is stolen from a database, it is stolen permanently.

7.3 ATM as a Multi-Factor Authentication Example

An Automated Teller Machine (ATM) is one of the most widely understood examples of multi-factor authentication in everyday life. To complete a transaction, the user must present:

- A physical bank card – this is a possession factor. The card contains a magnetic stripe or embedded chip that proves the user is in physical possession of the authorised payment instrument.
- A Personal Identification Number (PIN) – this is a knowledge factor. Only the account holder is supposed to know the four- to six-digit PIN associated with their card.

The combination of two independent factors significantly raises the bar for an attacker. Even if an attacker steals the card, they cannot complete a cash withdrawal without the PIN. Even if they observe or guess the PIN, they cannot use it without the physical card. This is the core security benefit of multi-factor authentication: compromise of any single factor is insufficient to grant access.

Nevertheless, the ATM model retains several notable residual risks:

- Card theft combined with shoulder surfing: An attacker who observes the user entering their PIN and then steals the card immediately has both factors. This is particularly common at busy ATMs or petrol stations.
- Card skimming: Criminals attach illegal card readers and miniature cameras to ATM card slots to clone the magnetic stripe data and capture PIN digits on video. The cloned card and captured PIN together satisfy both factors.
- SIM-swap or social engineering: Some banks use SMS OTP as a second factor for online banking. An attacker who convinces the mobile operator to redirect the victim's number can intercept the OTP.
- Malware and man-in-the-middle attacks: Online equivalents of ATM authentication can be defeated by malware that modifies what the user sees on-screen while silently altering the transaction data sent to the bank.

These remaining risks demonstrate that multi-factor authentication substantially increases security but does not make a system completely impenetrable. Layering additional controls – transaction limits, anomaly detection, biometric verification, and customer education – is necessary to further reduce residual risk.

9. Conclusion

This laboratory exercise provided comprehensive hands-on exposure to user authentication mechanisms on a Linux system. Through the creation of local user accounts, examination of the `/etc/passwd` and `/etc/shadow` files, and enforcement of a stricter password policy via `libpam-pwquality`, the practical link between system configuration choices and real-world security outcomes became clear: a misconfigured or default password policy can expose an organisation to trivially preventable account compromises.

The configuration and testing of SSH key-based authentication demonstrated why asymmetric cryptography is the preferred authentication mechanism for remote access in modern cyber security practice. Unlike passwords, private keys cannot be phished, are not vulnerable to brute-force attack within practical timeframes, and never traverse the network – making them fundamentally more secure for administrative access.

The analysis of authentication factors (knowledge, possession, and inherence) and the examination of the ATM as a real-world multi-factor system reinforced the principle that no single authentication mechanism is sufficient against a determined adversary. Layering independent factors – and understanding the residual risks in each combination – is the foundation of a mature identity and access management strategy.

The primary security lesson from this unit is that authentication strength is not a product feature that comes enabled by default: it must be deliberately configured, tested, and maintained. Administrators who understand both the technical controls and the threat model they are designed to address are essential to building resilient information systems.

10. References

- Lungu, N. (2025). *CYS 121 Information Systems Security – Lab Manual Unit 1: User Authentication*. Unpublished laboratory manual.
- National Institute of Standards and Technology. (2017). *Digital Identity Guidelines (NIST SP 800-63B)*. U.S. Department of Commerce.
- The Open Group. (2022). *PAM – Pluggable Authentication Modules for Linux*. Linux-PAM documentation.
- OpenSSH Project. (2024). *OpenSSH Manual Pages*. <https://www.openssh.com/manual.html>